

Backend

File Path – Doctor Appointment System/Backend/controllers/adminController.js

File Name – adminController.js

Description-Handles admin authentication, doctor management, appointments, and dashboard data securely.

Important Functions

(a) loginAdmin()

```
const loginAdmin = async (req,res)=>{
  const {email,password}=req.body;
  if(email===process.env.ADMIN_EMAIL && password===process.env.ADMIN_PASSWORD){
    const token=jwt.sign(email+password,process.env.JWT_SECRET);
    res.json({success:true,token});
  }else res.json({success:false,message:"Invalid credentials"});
}
```

(b) appointmentsAdmin()

```
const appointmentsAdmin = async (req,res)=>{
  const appointments=await appointmentModel.find({});
  res.json({success:true,appointments});
}
```

(c) appointmentCancel()

```
const appointmentCancel = async (req,res)=>{
  const {appointmentId}=req.body;
  await appointmentModel.findByIdAndUpdate(appointmentId,{cancelled:true});
  res.json({success:true,message:'Appointment Cancelled'});
}
```

(d) addDoctor()

```
const addDoctor = async (req,res)=>{
  const {name,email,password,speciality,degree,experience,about,fees,address}=req.body;
  const hash=await bcrypt.hash(password,10);
  const img=await cloudinary.uploader.upload(req.file.path);
  const newDoctor=new doctorModel({
    name,email,image:img.secure_url,password:hash,speciality,degree,experience,about,fees,
    address:JSON.parse(address),date:Date.now()});
  await newDoctor.save();
  res.json({success:true,message:'Doctor Added'});
}
```

(e) allDoctors()

```
const allDoctors = async (req,res)=>{
  const doctors=await doctorModel.find({}).select('-password');
  res.json({success:true,doctors});
}
```

(f) adminDashboard()

```
const adminDashboard = async (req,res)=>{
  const d=await doctorModel.find({}),u=await userModel.find({}),a=await appointmentModel.find({});
  res.json({success:true,dashData:{
    doctors:d.length,appointments:a.length,patients:u.length,latestAppointments:a.reverse() }});
}
```

Overall Functionality

This module serves as the core admin controller, enabling secure login, doctor onboarding, appointment control, and real-time dashboard insights. It ensures efficient data handling, security, and smooth system administration.

File Path – Doctor Appointment System/Backend/controllers/doctorController.js
File Name – doctorController.js

Description

Manages doctor login and appointment operations.

Important Functions

(a) loginDoctor()

```
const loginDoctor = async (req,res)=>{
  const {email,password}=req.body;
  const user=await doctorModel.findOne({email});
  if(!user) return res.json({success:false,message:"Invalid credentials"});
  const isMatch=await bcrypt.compare(password,user.password);
  if(isMatch){
    const token=jwt.sign({id:user._id},process.env.JWT_SECRET);
    res.json({success:true,token});
  } else res.json({success:false,message:"Invalid credentials"});
}
```

(b) appointmentsDoctor()

```
const appointmentsDoctor = async (req,res)=>{
  const {docId}=req.body;
  const appointments=await appointmentModel.find({docId});
  res.json({success:true,appointments});
}
```

(c) appointmentCancel()

```
const appointmentCancel = async (req,res)=>{
  const {docId,appointmentId}=req.body;
  const a=await appointmentModel.findById(appointmentId);
  if(a && a.docId===docId){
    await appointmentModel.findByIdAndUpdate(appointmentId,{cancelled:true});
    return res.json({success:true,message:'Appointment Cancelled'});
  }
  res.json({success:false,message:'Appointment Cancelled'});
}
```

(d) appointmentComplete()

```
const appointmentComplete = async (req,res)=>{
  const {docId,appointmentId}=req.body;
  const a=await appointmentModel.findById(appointmentId);
  if(a && a.docId===docId){
    await appointmentModel.findByIdAndUpdate(appointmentId,{isCompleted:true});
    return res.json({success:true,message:'Appointment Completed'});
  }
  res.json({success:false,message:'Appointment Cancelled'});
}
```

Overall Functionality

Handles doctor authentication and appointment management securely.

File Path – Doctor Appointment System/Backend/controllers/doctorController.js
File Name – doctorController.js

Description

Manages doctor profile, availability, and dashboard data.

Important Functions

(e) doctorList()

```
const doctorList = async (req,res)=>{
  const doctors=await doctorModel.find({}).select(['-password','-email']);
  res.json({success:true,doctors});
}
```

(f) changeAvailability()

```
const changeAvailability = async (req,res)=>{
  const {docId}=req.body;
  const d=await doctorModel.findById(docId);
  await doctorModel.findByIdAndUpdate(docId,{available:!d.available});
  res.json({success:true,message:'Availability Changed'});
}
```

(g) doctorProfile()

```
const doctorProfile = async (req,res)=>{
  const {docId}=req.body;
  const profileData=await doctorModel.findById(docId).select('-password');
  res.json({success:true,profileData});
}
```

(h) updateDoctorProfile()

```
const updateDoctorProfile = async (req,res)=>{
  const {docId,fees,address,available}=req.body;
  await doctorModel.findByIdAndUpdate(docId,{fees,address,available});
  res.json({success:true,message:'Profile Updated'});
}
```

(i) doctorDashboard()

```
const doctorDashboard = async (req,res)=>{
  const {docId}=req.body;
  const a=await appointmentModel.find({docId});
  let earnings=0,patients=[];
  a.map(i=>{if(i.isCompleted||i.payment) earnings+=i.amount});
  a.map(i=>{if(!patients.includes(i.userId)) patients.push(i.userId)});
  res.json({success:true,dashData:{
    earnings,appointments:a.length,patients:patients.length,latestAppointments:a.reverse()
  }});
}
```

Overall Functionality

Handles doctor profile, availability, and performance analytics.

File Path – Doctor Appointment System/Backend/controllers/userController.js
File Name – userController.js

Description

Handles user authentication and profile management.

Important Functions

(a) registerUser()

```
const registerUser = async (req,res)=>{
  const {name,email,password}=req.body;
  if(!name || !email || !password) return res.json({success:false,message:'Missing Details'});
  if(!validator.isEmail(email)) return res.json({success:false,message:'Invalid Email'});
  if(password.length<8) return res.json({success:false,message:'Weak Password'});
  const hash=await bcrypt.hash(password,10);
  const user=await new userModel({name,email,password:hash}).save();
  const token=jwt.sign({id:user._id},process.env.JWT_SECRET);
  res.json({success:true,token});
}
```

(b) loginUser()

```
const loginUser = async (req,res)=>{
  const {email,password}=req.body;
  const user=await userModel.findOne({email});
  if(!user) return res.json({success:false,message:'User not found'});
  const isMatch=await bcrypt.compare(password,user.password);
  if(isMatch){
    const token=jwt.sign({id:user._id},process.env.JWT_SECRET);
    res.json({success:true,token});
  }else res.json({success:false,message:'Invalid credentials'});
}
```

(c) getProfile()

```
const getProfile = async (req,res)=>{
  const {userId}=req.body;
  const userData=await userModel.findById(userId).select('-password');
  res.json({success:true,userData});
}
```

(d) updateProfile()

```
const updateProfile = async (req,res)=>{
  const {userId,name,phone,address,dob,gender}=req.body;
  await userModel.findByIdAndUpdate(userId,{name,phone,address:JSON.parse(address),dob,gender});
  if(req.file){
    const img=await cloudinary.uploader.upload(req.file.path);
    await userModel.findByIdAndUpdate(userId,{image:img.secure_url});
  }
  res.json({success:true,message:'Profile Updated'});
}
```

Overall Functionality

Handles user registration, login, and profile updates securely.

File Path – Doctor Appointment System/Backend/middleware/authAdmin.js

File Name – authAdmin.js

Description

Handles admin authentication using JWT token validation.

Important Functions

(a) authAdmin()

```
const authAdmin = async (req,res,next)=>{
  try{
    const {atoken}=req.headers;
    if(!atoken) return res.json({success:false,message:'Not Authorized Login Again'});
    const decode=jwt.verify(atoken,process.env.JWT_SECRET);
    if(decode!==process.env.ADMIN_EMAIL+process.env.ADMIN_PASSWORD)
      return res.json({success:false,message:'Not Authorized Login Again'});
    next();
  }catch(e){
    res.json({success:false,message:e.message});
  }
}
```

Overall Functionality

Ensures only authorized admin can access protected routes.

File Path – Doctor Appointment System/Backend/middleware/authDoctor.js

File Name – authDoctor.js

Description

Handles doctor authentication using JWT token.

Important Functions

(a) authDoctor()

```
const authDoctor = async (req,res,next)=>{
  const {dtoken}=req.headers;
  if(!dtoken) return res.json({success:false,message:'Not Authorized Login Again'});
  try{
    const decode=jwt.verify(dtoken,process.env.JWT_SECRET);
    req.body.docId=decode.id;
    next();
  }catch(e){
    res.json({success:false,message:e.message});
  }
}
```

Overall Functionality

Ensures only authenticated doctors can access protected routes.

File Path – Doctor Appointment System/Backend/middleware/authUser.js

File Name – authUser.js

Description

Handles user authentication using JWT token.

Important Functions

(a) authUser()

```
const authUser = async (req,res,next)=>{
  const {token}=req.headers;
  if(!token) return res.json({success:false,message:'Not Authorized Login Again'});
  try{
    const decode=jwt.verify(token,process.env.JWT_SECRET);
    req.body.userId=decode.id;
    next();
  }catch(e){
    res.json({success:false,message:e.message});
  }
}
```

Overall Functionality

Ensures only authenticated users can access protected routes.

File Path – Doctor Appointment System/Backend/middleware/multer.js

File Name – multer.js

Description

Handles file upload configuration using multer.

Important Functions

(a) multerStorage()

```
const storage = multer.diskStorage({  
  filename:(req,file,cb)=>{  
    cb(null,file.originalname);  
  }  
});
```

(b) uploadMiddleware

```
const upload = multer({storage:storage});
```

Overall Functionality

Configures file uploads and stores files with original names on server.

File Path – Doctor Appointment System/Backend/routes/adminRoutes.js

File Name – adminRoutes.js

Description

Defines admin routes for authentication, doctor management, and appointments.

Important Functions

(a) adminRouter Setup

```
const adminRouter = express.Router();
```

(b) login Route

```
adminRouter.post("/login", loginAdmin);
```

(c) addDoctor Route

```
adminRouter.post("/add-doctor", authAdmin, upload.single('image'), addDoctor);
```

(d) appointments Route

```
adminRouter.get("/appointments", authAdmin, appointmentsAdmin);
```

(e) cancelAppointment Route

```
adminRouter.post("/cancel-appointment", authAdmin, appointmentCancel);
```

(f) allDoctors Route

```
adminRouter.get("/all-doctors", authAdmin, allDoctors);
```

(g) changeAvailability Route

```
adminRouter.post("/change-availability", authAdmin, changeAvailability);
```

(h) dashboard Route

```
adminRouter.get("/dashboard", authAdmin, adminDashboard);
```

Overall Functionality

Manages all admin API routes with authentication and middleware integration.

File Path – Doctor Appointment System/Backend/routes/doctorRoutes.js
File Name – doctorRoutes.js

Description

Defines doctor routes for login, appointments, profile, and dashboard.

Important Functions

(a) doctorRouter Setup

```
const doctorRouter = express.Router();
```

(b) login Route

```
doctorRouter.post("/login", loginDoctor);
```

(c) cancelAppointment Route

```
doctorRouter.post("/cancel-appointment", authDoctor, appointmentCancel);
```

(d) appointments Route

```
doctorRouter.get("/appointments", authDoctor, appointmentsDoctor);
```

(e) doctorList Route

```
doctorRouter.get("/list", doctorList);
```

(f) changeAvailability Route

```
doctorRouter.post("/change-availability", authDoctor, changeAvailability);
```

(g) completeAppointment Route

```
doctorRouter.post("/complete-appointment", authDoctor, appointmentComplete);
```

(h) dashboard Route

```
doctorRouter.get("/dashboard", authDoctor, doctorDashboard);
```

(i) profile Route

```
doctorRouter.get("/profile", authDoctor, doctorProfile);
```

(j) updateProfile Route

```
doctorRouter.post("/update-profile", authDoctor, updateDoctorProfile);
```

Overall Functionality

Manages doctor API routes with authentication and access control.

File Path – Doctor Appointment System/Backend/routes/userRoutes.js

File Name – userRoutes.js

Description

Defines user routes for authentication, profile, appointments, and payments.

Important Functions

(a) userRouter Setup

```
const userRouter = express.Router();
```

(b) auth Routes

```
userRouter.post("/register", registerUser);  
userRouter.post("/login", loginUser);
```

(c) profile Routes

```
userRouter.get("/get-profile", authUser, getProfile);  
userRouter.post("/update-profile", upload.single('image'), authUser, updateProfile);
```

(d) appointment Routes

```
userRouter.post("/book-appointment", authUser, bookAppointment);  
userRouter.get("/appointments", authUser, listAppointment);  
userRouter.post("/cancel-appointment", authUser, cancelAppointment);
```

(e) payment Routes (Razorpay)

```
userRouter.post("/payment-razorpay", authUser, paymentRazorpay);  
userRouter.post("/verifyRazorpay", authUser, verifyRazorpay);
```

(f) payment Routes (Stripe)

```
userRouter.post("/payment-stripe", authUser, paymentStripe);  
userRouter.post("/verifyStripe", authUser, verifyStripe);
```

Overall Functionality

Manages user APIs including authentication, profile handling, appointments, and payment integration.

Frontend

File Path – Doctor Appointment System/Frontend/src/components/Banner.jsx

File Name – Banner.jsx

Description

Displays homepage banner with CTA to book appointments.

Important Functions

(a) Banner Component

```
const Banner = () => {  
  const navigate = useNavigate();  
  
  return (  
    <div className='flex bg-primary rounded-lg px-6 md:px-14 my-20'>  
  
      <div className='flex-1 py-10 text-white'>  
        <p className='text-3xl font-semibold'>Book Appointment</p>  
        <p className='mt-2'>With 100+ Trusted Doctors</p>  
        <button onClick={()=>{navigate('/login');scrollTo(0,0)}}  
          className='bg-white text-sm px-6 py-2 rounded-full mt-4'>  
          Create account  
        </button>  
      </div>  
  
      <div className='hidden md:block w-1/2 relative'>  
        <img className='absolute bottom-0 right-0'  
          src={assets.appointment_img} alt=''/>  
      </div>  
  
    </div>  
  );  
}
```

Overall Functionality

Provides a visually appealing banner with navigation to login/signup page.

File Path – Doctor Appointment System/Frontend/src/components/Footer.jsx
File Name – Footer.jsx

Description

Displays website footer with company info and contact details.

Important Functions

(a) Footer Component

```
const Footer = () => {  
  return (  
    <div className='md:mx-10'>  
  
      <div className='grid grid-cols-[3fr_1fr_1fr] gap-10 my-10 text-sm'>  
  
        <div>  
          <img src={assets.logo} className='w-40 mb-3' />  
          <p className='text-gray-600'>Company description text...</p>  
        </div>  
  
        <div>  
          <p className='font-medium mb-3'>COMPANY</p>  
          <ul className='text-gray-600'>  
            <li>Home</li><li>About</li><li>Privacy</li>  
          </ul>  
        </div>  
  
        <div>  
          <p className='font-medium mb-3'>CONTACT</p>  
          <ul className='text-gray-600'>  
            <li>+1-212-456-7890</li>  
            <li>email@gmail.com</li>  
          </ul>  
        </div>  
  
      </div>  
  
      <hr />  
      <p className='text-center text-sm py-3'>© 2024 Prescripto</p>  
  
    </div>  
  );  
}
```

Overall Functionality

Provides footer layout with company details, links, and contact information.

File Path – Doctor Appointment System/Frontend/src/components/Header.jsx

File Name – Header.jsx

Description

Displays homepage header with CTA and doctor highlights.

Important Functions

(a) Header Component

```
const Header = () => {  
  return (  
    <div className='flex md:flex-row bg-primary rounded-lg px-6 md:px-10'>  
  
      <div className='md:w-1/2 flex flex-col justify-center text-white py-10'>  
        <p className='text-4xl font-semibold'>  
          Book Appointment <br/> With Trusted Doctors  
        </p>  
  
        <div className='flex items-center gap-2 text-sm mt-3'>  
          <img className='w-20' src={assets.group_profiles}/>  
          <p>Browse doctors & book easily</p>  
        </div>  
  
        <a href='#speciality'  
          className='bg-white text-sm px-6 py-2 rounded-full mt-4'>  
          Book appointment  
        </a>  
      </div>  
  
      <div className='md:w-1/2 relative'>  
        <img className='w-full md:absolute bottom-0'  
          src={assets.header_img}/>  
      </div>  
  
    </div>  
  );  
}
```

Overall Functionality

Provides a header section with introduction and quick appointment navigation.

File Path – Doctor Appointment System/Frontend/src/components/Navbar.jsx

File Name – Navbar.jsx

Description

Handles navigation, user menu, and authentication UI.

Important Functions

(a) Navbar Component

```
const Navbar = () => {
  const navigate = useNavigate();
  const [showMenu, setShowMenu] = useState(false);
  const {token, setToken, userData} = useContext(AppContext);

  const logout = () => {
    localStorage.removeItem('token');
    setToken(false);
    navigate('/login');
  };

  return (
    <div className='flex justify-between py-4 border-b'>

      <img onClick={()=>navigate('/')} src={assets.logo} className='w-32' />

      <ul className='hidden md:flex gap-4'>
        <NavLink to='/'><li>HOME</li></NavLink>
        <NavLink to='/doctors'><li>DOCTORS</li></NavLink>
        <NavLink to='/about'><li>ABOUT</li></NavLink>
        <NavLink to='/contact'><li>CONTACT</li></NavLink>
      </ul>

      <div className='flex items-center gap-3'>
        {token ? (
          <div className='relative group'>
            <img src={userData.image} className='w-8 rounded-full' />
            <div className='absolute hidden group-hover:block bg-white p-3'>
              <p onClick={()=>navigate('/my-profile')}>Profile</p>
              <p onClick={()=>navigate('/my-appointments')}>Appointments</p>
              <p onClick={logout}>Logout</p>
            </div>
          </div>
        ) : (
          <button onClick={()=>navigate('/login')}
            className='bg-primary text-white px-4 py-2 rounded-full'>
            Login
          </button>
        )}

        <img onClick={()=>setShowMenu(true)} src={assets.menu_icon} className='w-5 md:hidden' />
      </div>

    </div>
  );
}
```

Overall Functionality

Provides navigation, authentication handling, and responsive menu UI.

File Path – Doctor Appointment System/Frontend/src/components/RelatedDoctors.jsx
File Name – RelatedDoctors.jsx

Description

Displays doctors with same speciality for recommendations.

Important Functions

(a) RelatedDoctors Component

```
const RelatedDoctors = ({speciality,docId})=>{
  const navigate=useNavigate();
  const {doctors}=useContext(AppContext);
  const [relDoc,setRelDoc]=useState([]);

  useEffect(()=>{
    if(doctors.length && speciality){
      setRelDoc(doctors.filter(d=>d.speciality===speciality && d._id!==docId));
    }
  },[doctors,speciality,docId]);

  return (
    <div className='text-center my-10'>
      <h1 className='text-2xl'>Related Doctors</h1>

      <div className='grid gap-4 mt-5'>
        {relDoc.map((d,i)=>(
          <div key={i}
            onClick={()=>{navigate(`/appointment/${d._id}`);scrollTo(0,0)}}
            className='border p-3 cursor-pointer'>

            <img src={d.image}/>
            <p>{d.name}</p>
            <p>{d.speciality}</p>
            <p>{d.available?'Available':'Not Available'}</p>

          </div>
        ))}
      </div>
    </div>
  );
}
```

Overall Functionality

Filters and displays related doctors based on speciality for better user suggestions.

File Path – Doctor Appointment System/Frontend/src/components/SpecialityMenu.jsx
File Name – SpecialityMenu.jsx

Description

Displays list of doctor specialities for easy navigation.

Important Functions

(a) SpecialityMenu Component

```
const SpecialityMenu = ()=>{
  return (
    <div id='speciality' className='text-center py-10'>

      <h1 className='text-2xl'>Find by Speciality</h1>
      <p className='text-sm'>Browse doctors by category</p>

      <div className='flex gap-3 mt-5 overflow-x-auto'>
        {specialityData.map((item,i)=>{
          <Link key={i}
            to={`/${doctors}/${item.speciality}`}
            onClick={()=>scrollTo(0,0)}
            className='flex flex-col items-center cursor-pointer'>

            <img src={item.image} className='w-16'/>
            <p>{item.speciality}</p>

          </Link>
        ))}
      </div>

    </div>
  );
}
```

Overall Functionality

Provides speciality-based navigation for selecting doctors easily.

File Path – Doctor Appointment System/Frontend/src/components/TopDoctors.jsx
File Name – TopDoctors.jsx

Description

Displays top doctors with quick booking option.

Important Functions

(a) TopDoctors Component

```
const TopDoctors = ()=>{
  const navigate=useNavigate();
  const {doctors}=useContext(AppContext);

  return (
    <div className='text-center my-10'>

      <h1 className='text-2xl'>Top Doctors</h1>

      <div className='grid gap-4 mt-5'>
        {doctors.slice(0,10).map((d,i)=>{
          <div key={i}
            onClick={()=>{navigate(`/appointment/${d._id}`);scrollTo(0,0)}}
            className='border p-3 cursor-pointer'>

            <img src={d.image}/>
            <p>{d.name}</p>
            <p>{d.speciality}</p>
            <p>{d.available?'Available':'Not Available'}</p>

          </div>
        ))}
      </div>

      <button onClick={()=>navigate('/doctors')}
        className='mt-4 px-6 py-2 bg-gray-200 rounded'>
        More
      </button>

    </div>
  );
}
```

Overall Functionality

Shows top doctors and allows quick navigation to booking and full doctor list.

File Path – Doctor Appointment System/Frontend/src/App.jsx

File Name – App.jsx

Description

Main app component handling routing and layout.

Important Functions

(a) App Component

```
const App = ()=>{
  return (
    <div className='mx-4 sm:mx-[10%]'>

      <ToastContainer/>
      <Navbar/>

      <Routes>
        <Route path='/' element={<Home/>}/>
        <Route path='/doctors' element={<Doctors/>}/>
        <Route path='/doctors/:speciality' element={<Doctors/>}/>
        <Route path='/login' element={<Login/>}/>
        <Route path='/about' element={<About/>}/>
        <Route path='/contact' element={<Contact/>}/>
        <Route path='/appointment/:docId' element={<Appointment/>}/>
        <Route path='/my-appointments' element={<MyAppointments/>}/>
        <Route path='/my-profile' element={<MyProfile/>}/>
        <Route path='/verify' element={<Verify/>}/>
      </Routes>

      <Footer/>

    </div>
  );
}
```

Overall Functionality

Controls routing, navigation, and overall layout of the application.

File Path – Doctor Appointment System/Frontend/src/index.css

File Name – index.css

Description

Defines global styles and Tailwind CSS configuration.

Important Functions

(a) Import & Tailwind Setup

```
@import url('https://fonts.googleapis.com/css2?family=Outfit:wght@100..900&display=swap');
```

```
@tailwind base;  
@tailwind components;  
@tailwind utilities;
```

(b) Global Styles

```
*{  
  font-family: Outfit;  
}
```

(c) Active State Styling

```
.active hr{  
  @apply block;  
}
```

```
@media(max-width:740px){  
  .active p{  
    @apply text-white bg-primary;  
  }  
}
```

(d) Scrollbar Styling

```
::-webkit-scrollbar{  
  @apply hidden;  
}
```

Overall Functionality

Applies global styling, font setup, responsive design, and UI enhancements.

File Path – Doctor Appointment System/Frontend/src/main.jsx

File Name – main.jsx

Description

Initializes React app with routing and global context.

Important Functions

(a) App Initialization

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <BrowserRouter>  
    <AppContextProvider>  
      <App/>  
    </AppContextProvider>  
  </BrowserRouter>  
);
```

Overall Functionality

Bootstraps the application with routing and global state management.